1. Red-Black - Tree Insertions

```
#include <stdio.h>
#include <stdlib.h>

enum color_type { R, B };

struct node {
int data;
enum color_type type;
struct node *left, *right, *parent;

};

struct node *root = NULL;
struct node *newnode = NULL;
struct node *current = NULL;
struct node *follow = NULL;

void createnode(int x);
void insertnode();
void doublered();
void case1(struct node *z);
void case2(struct node *z);
void traversal();

int main() {
   int n, x;
   char op;

   scanf("%d",&n);

   for(int i =0;i<n;i++){
      scanf(" %c %d",&op,&x);
      if(op == 'I'){
         createnode(x);
         insertnode();
         doublered();
```

```
        }
    }
    traversal();
}
void createnode(int x){
    newnode = malloc(sizeof(struct node));
    newnode->data = x;
    newnode->left = newnode->right = newnode->parent = NULL;

    if(root == NULL)
        newnode->type = B;
    else
        newnode->type = R;
}

void insertnode(){

    if(root == NULL){
        root = newnode;
        return;
    }

    current = root;
    follow = NULL;

    while(current != NULL){
        follow = current;
        if(newnode->data < current->data)
            current = current->left;
        else
            current = current->right;
    }
    newnode->parent = follow;

    if(newnode->data < follow->data)
        follow->left = newnode;
    else
```

```
        follow->right = newnode;
}

void doublered(){

    struct node *z = newnode;

    while(z != root && z->parent->type == R){

        struct node *p = z->parent;
        struct node *g = p->parent;
        if(!g) break;

        struct node *u = (p == g->left)? g->right : g->left;

        if(u && u->type == R){
            case1(z);
        }
        else{
            case2(z);
            break;
        }

        z = g;
    }

    root->type = B;
}

void case1(struct node *z){
    struct node *p = z->parent;
    struct node *g = p->parent;
    struct node *u = (p == g->left)? g->right : g->left;

    p->type =B;
    if(u) u->type = B;
    g->type = R;
```

```
}

void case2(struct node *z){
    struct node *p = z->parent;
    struct node *g = p->parent;

    if(p == g->left){
        if(z == p->right){
            p->right = z->left;
            if(z->left) z->left->parent = p;
            z->left = p;
            p->parent = z;
            g->left = z;
            z->parent = g;
            p = z;
            g = p->parent;
        }
        g->left = p->right;
        if(p->right) p->right->parent = g;
        p->right = g;
    }
    else{
        if(z == p->left){
            p->left = z->right;
            if(z->right) z->right->parent = p;
            z->right = p;
            p->parent = z;
            g->right = z;
            z->parent = g;
            p = z;
            g = p->parent;

        }

        g->right = p->left;
        if(p->left) p->left->parent = g;
        p->left = g;
```

```
    }
    p->parent = g->parent;

    if(!g->parent)
        root = p;
    else if(g == g->parent->left)
        g->parent->left = p;
    else
        g->parent->right = p;

    g->parent = p;

    p->type = B;
    g->type = R;
}

void traversal(){
    if(!root) return;

    struct node* q[100];
    int f=0,r=0;
    q[r++] = root;

    while(f<r){
        int size = r-f;
        while(size -- ){
            struct node* t = q[f++];
            printf("%d(%c) ", t->data, t->type == R?'R':'B');  **Idhar space ka dhyan rakho X aata hai if nhi dekha toh

            if(t->left) q[r++] = t->left;
            if(t->right) q[r++] = t->right;
        }
        printf("\n");
    }
}
```

2. Fractional Knapsack Problem

```
#include <stdio.h>

struct Item {
    int value;
    int weight;
    float ratio;
};

void sortRatio(struct Item items[], int n) {
    struct Item temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (items[i].ratio < items[j].ratio) {
                temp = items[i];
                items[i] = items[j];
                items[j] = temp;
            }
        }
    }
}

int main() {
    int n;
    float W;

    scanf("%d", &n);
    scanf("%f", &W);

    struct Item items[n];

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &items[i].value, &items[i].weight);
        items[i].ratio = (float)items[i].value / items[i].weight;
    }

    sortRatio(items, n);
```

```
    float totalProfit = 0.0;
    float balance = W;
    float fraction;

    for (int i = 0; i < n && balance > 0; i++) {
        if (items[i].weight <= balance) {
            totalProfit = totalProfit + items[i].value;
            balance = balance - items[i].weight;
        }
        else {
            fraction = balance / items[i].weight;
            totalProfit = totalProfit + items[i].value * fraction;
            balance = 0;
        }
    }

    printf("%.2f\n", totalProfit);

    return 0;
}
```

3. Job Sequencing with Deadlines

```
#include <stdio.h>

struct Task {
    int id;
    int deadline;
    int profit;
};

void sortProfit(struct Task t[], int n) {
    struct Task temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (t[i].profit < t[j].profit) {
                temp = t[i];
                t[i] = t[j];
                t[j] = temp;
            }
        }
    }
}

int findMaxDeadline(struct Task t[], int n) {
    int max = 0;
    for (int i = 0; i < n; i++) {
        if (t[i].deadline > max) {
            max = t[i].deadline;
        }
    }
    return max;
}

int main() {
    int n;
    scanf("%d", &n);

    struct Task tasks[n];
```

```
for (int i = 0; i < n; i++) {
   scanf("%d %d %d",&tasks[i].id, &tasks[i].deadline, &tasks[i].profit);
}

sortProfit(tasks, n);

int maxDeadline = findMaxDeadline(tasks, n);

int slot[maxDeadline];
for (int i = 0; i < maxDeadline; i++) {
   slot[i] = -1;
}

int totalProfit = 0;

for (int i = 0; i < n; i++) {
   int index = tasks[i].deadline - 1;

   while (index >= 0) {
      if (slot[index] == -1) {
         slot[index] = tasks[i].id;
         totalProfit = totalProfit + tasks[i].profit;
         break;
      }
      index--;
   }
}

printf("Scheduled tasks: ");
for (int i = 0; i < maxDeadline; i++) {
   if (slot[i] != -1) {
      printf("%d ", slot[i]);
   }
}
printf("\nMaximum Profit: %d\n", totalProfit);
```

```
    return 0;
}
```

4. Coin Change Problem

```
#include <stdio.h>
#include <limits.h>

int minCoins(int coins[], int n, int target) {
    int dp[target + 1];

    // Initialize dp array
    for (int i = 0; i <= target; i++) {
        dp[i] = INT_MAX;
    }

    dp[0] = 0; // Base case

    // Build dp array
    for (int i = 1; i <= target; i++) {
        for (int j = 0; j < n; j++) {
            if (coins[j] <= i && dp[i - coins[j]] != INT_MAX) {
                int res = dp[i - coins[j]] + 1;
                if (res < dp[i]) {
                    dp[i] = res;
                }
            }
        }
    }

    // If not possible
    if (dp[target] == INT_MAX) {
        return -1;
    }

    return dp[target];
}

int main() {
    int n, target;
```

```
    printf("Enter number of denominations: ");
    scanf("%d", &n);

    int coins[n];
    printf("Enter coin denominations: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &coins[i]);
    }

    printf("Enter target amount: ");
    scanf("%d", &target);

    int result = minCoins(coins, n, target);

    if (result == -1) {
        printf("Minimum coins required: -1\n");
    } else {
        printf("Minimum coins required: %d\n", result);
    }

    return 0;
}
```

5. Matrix Chain Multiplication

```
#include <stdio.h>
#include <limits.h>

int matrixChainOrder(int p[], int n) {
   int m[n][n];

   for (int i = 1; i < n; i++)
      m[i][i] = 0;

   for (int L = 2; L < n; L++) {
      for (int i = 1; i < n - L + 1; i++) {
         int j = i + L - 1;
         m[i][j] = INT_MAX;

         for (int k = i; k < j; k++) {
            int q = m[i][k] + m[k + 1][j]
                  + p[i - 1] * p[k] * p[j];

            if (q < m[i][j])
               m[i][j] = q;
         }
      }
   }

   return m[1][n - 1];
}

int main() {
   int n;

   scanf("%d", &n);

   int p[n + 1];
   for (int i = 0; i <= n; i++) {
      scanf("%d", &p[i]);
   }
```

```
    int result = matrixChainOrder(p, n + 1);

    printf("%d\n", result);

    return 0;
}
```

6. Rabin–Karp string matching algorithm

```
#include <stdio.h>
#include <string.h>

#define d 256
#define q 101

void rabinKarp(char text[], char pattern[])
{
   int n = strlen(text);
   int m = strlen(pattern);
   int i, j;
   int p = 0;   // hash value for pattern
   int t = 0;   // hash value for text
   int h = 1;
   int found = 0;

   for(i = 0; i < m-1; i++)
      h = (h * d) % q;

   for(i = 0; i < m; i++)
   {
      p = (d * p + pattern[i]) % q;
      t = (d * t + text[i]) % q;
   }

   for(i = 0; i <= n - m; i++)
   {
      if(p == t)
      {
         for(j = 0; j < m; j++)
         {
            if(text[i+j] != pattern[j])
               break;
         }

         if(j == m)
```

```
            {
                printf("Pattern found at index: %d\n", i);
                found = 1;
            }
        }

        if(i < n-m)
        {
            t = (d * (t - text[i] * h) + text[i+m]) % q;

            if(t < 0)
                t = t + q;
        }
    }

    if(found == 0)
        printf("Pattern not found\n");
}

int main()
{
    char text[100], pattern[100];

    printf("Enter text: ");
    scanf("%s", text);

    printf("Enter pattern: ");
    scanf("%s", pattern);

    rabinKarp(text, pattern);

    return 0;
}
```

7. Longest Common Subsequence (LCS)

```
#include <stdio.h>
#include <string.h>

int c[100][100];
char arrow[100][100];

void printLCS(char arrow[][100], char X[], int i, int j)
{
   if (i == 0 || j == 0)
      return;

   if (arrow[i][j] == 'D')
   {
      printLCS(arrow, X, i-1, j-1);
      printf("%c", X[i-1]);
   }
   else if (arrow[i][j] == 'U')
   {
      printLCS(arrow, X, i-1, j);
   }
   else
   {
      printLCS(arrow, X, i, j-1);
   }
}

int main()
{
   char X[100], Y[100];
   int m, n, i, j;

   scanf("%s", X);
   scanf("%s", Y);

   m = strlen(X);
   n = strlen(Y);
```

```
    for(i = 0; i <= m; i++)
    {
        for(j = 0; j <= n; j++)
        {
            if(i == 0 || j == 0)
            {
                c[i][j] = 0;
            }
            else if(X[i-1] == Y[j-1])
            {
                c[i][j] = c[i-1][j-1] + 1;
                arrow[i][j] = 'D';
            }
            else if(c[i-1][j] >= c[i][j-1])
            {
                c[i][j] = c[i-1][j];
                arrow[i][j] = 'U';
            }
            else
            {
                c[i][j] = c[i][j-1];
                arrow[i][j] = 'L';
            }
        }
    }

    printf("Length of LCS: %d\n", c[m][n]);
    printf("LCS: ");
    printLCS(arrow, X, m, n);

    return 0;
}
```

8. Computational Efficiency Analyzer

```
#include <stdio.h>

// Recursive function
int fib_recursive(int n) {
    if (n <= 1)
        return n;
    return fib_recursive(n - 1) + fib_recursive(n - 2);
}

// Iterative function
int fib_iterative(int n) {
    int a = 0, b = 1, c;

    if (n == 0)
        return 0;

    for (int i = 2; i <= n; i++) {
        c = a + b;
        a = b;
        b = c;
    }

    return b;
}

int main() {
    int n;

    printf("Enter N: ");
    scanf("%d", &n);

    printf("Recursive Fibonacci(%d): %d\n", n, fib_recursive(n));
    printf("Non-Recursive Fibonacci(%d): %d\n", n, fib_iterative(n));

    return 0;
}
```

Tower of Hanoi algorithm

```
#include <stdio.h>

// Function to solve Tower of Hanoi
void toh(int n, char source, char auxiliary, char destination)
{
   // Base case
   if (n == 1)
   {
      printf("Move disk 1 from %c to %c\n", source, destination);
      return;
   }

   // Move n-1 disks from source to auxiliary
   toh(n - 1, source, destination, auxiliary);

   // Move nth disk from source to destination
   printf("Move disk %d from %c to %c\n", n, source, destination);

   // Move n-1 disks from auxiliary to destination
   toh(n - 1, auxiliary, source, destination);
}

int main()
{
   int n;

   printf("Enter number of disks: ");
   scanf("%d", &n);

   // Call function
   toh(n, 'A', 'B', 'C');

   return 0;
}
```

Quick Sort

```
#include <stdio.h>

// Function to swap two elements
void swap(int *a, int *b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}

// Partition function
int partition(int arr[], int low, int high)
{
   int pivot = arr[high];  // choosing last element as pivot
   int i = low - 1;

   for (int j = low; j < high; j++)
   {
      if (arr[j] < pivot)
      {
         i++;
         swap(&arr[i], &arr[j]);
      }
   }

   swap(&arr[i + 1], &arr[high]);
   return i + 1;
}

// Quick Sort function
void quickSort(int arr[], int low, int high)
{
   if (low < high)
   {
      int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main()
{
    int n, i, arr[100];

    printf("Enter number of products: ");
    scanf("%d", &n);

    printf("Enter product prices:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    printf("Sorted prices:\n");
    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

0/1 Knapsack Problem

```
#include <stdio.h>

// Function to find maximum value
int max(int a, int b)
{
   return (a > b) ? a : b;
}

int main()
{
   int n, W, i, w;

   printf("Enter number of items: ");
   scanf("%d", &n);

   int value[100], weight[100];

   printf("Enter values:\n");
   for (i = 0; i < n; i++)
      scanf("%d", &value[i]);

   printf("Enter weights:\n");
   for (i = 0; i < n; i++)
      scanf("%d", &weight[i]);

   printf("Enter maximum capacity: ");
   scanf("%d", &W);

   int dp[101][101];

   // Build DP table
   for (i = 0; i <= n; i++)
   {
      for (w = 0; w <= W; w++)
      {
         if (i == 0 || w == 0)
```

```
            dp[i][w] = 0;
        else if (weight[i - 1] <= w)
            dp[i][w] = max(value[i - 1] + dp[i - 1][w - weight[i - 1]],
                           dp[i - 1][w]);
        else
            dp[i][w] = dp[i - 1][w];
      }
   }

   printf("Maximum value = %d", dp[n][W]);

   return 0;
}
```

Binary Search algorithm(4 - 5)

```
#include <stdio.h>

// Function to check if array is sorted (ascending)
int isSorted(int arr[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        if (arr[i] > arr[i + 1])
            return 0;
    }
    return 1;
}

// Binary Search function
int binarySearch(int arr[], int n, int key)
{
    int low = 0, high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main()
{
    int n, arr[100], key;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("Enter element to search: ");
    scanf("%d", &key);

    // Check if sorted
    if (!isSorted(arr, n))
    {
        printf("Array is not sorted");
        return 0;
    }

    int result = binarySearch(arr, n, key);

    if (result != -1)
        printf("%d", result);
    else
        printf("-1");

    return 0;
}
```

Merge Sort

```
#include <stdio.h>

// Function to merge two subarrays
void merge(int arr[], int left, int mid, int right)
{
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[100], R[100];

    // Copy data to temp arrays
    for (i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    i = 0;
    j = 0;
    k = left;

    // Merge the temp arrays
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
```

```
    // Copy remaining elements
    while (i < n1)
    {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2)
    {
        arr[k] = R[j];
        j++;
        k++;
    }
}

// Merge Sort function
void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main()
{
    int n, arr[100], i;

    printf("Enter number of transactions: ");
    scanf("%d", &n);
```

```
    printf("Enter transaction amounts:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    mergeSort(arr, 0, n - 1);

    printf("Sorted transactions:\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

Min-Max algorithm

```
#include <stdio.h>

// Structure to store min and max
struct Pair
{
   int min;
   int max;
};

// Function to find min and max using divide and conquer
struct Pair findMinMax(int arr[], int low, int high)
{
   struct Pair result, left, right;

   // Base case: only one element
   if (low == high)
   {
      result.min = arr[low];
      result.max = arr[low];
      return result;
   }

   // Base case: two elements
   if (high == low + 1)
   {
      if (arr[low] < arr[high])
      {
         result.min = arr[low];
         result.max = arr[high];
      }
      else
      {
         result.min = arr[high];
         result.max = arr[low];
      }
      return result;
```

```
    }

    // Divide array into two halves
    int mid = (low + high) / 2;

    left = findMinMax(arr, low, mid);
    right = findMinMax(arr, mid + 1, high);

    // Combine results
    result.min = (left.min < right.min) ? left.min : right.min;
    result.max = (left.max > right.max) ? left.max : right.max;

    return result;
}

int main()
{
    int n, arr[100], i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter stock prices:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    struct Pair ans = findMinMax(arr, 0, n - 1);

    printf("Minimum: %d\n", ans.min);
    printf("Maximum: %d\n", ans.max);

    return 0;
}
```

Optimal Binary Search Tree (OBST) algorithm

```
#include <stdio.h>

int min(int a, int b)
{
   return (a < b) ? a : b;
}

// Function to calculate sum of frequencies
int sum(int freq[], int i, int j)
{
   int s = 0;
   for (int k = i; k <= j; k++)
      s += freq[k];
   return s;
}

int main()
{
   int n;

   printf("Enter number of keys: ");
   scanf("%d", &n);

   int keys[100], freq[100];

   printf("Enter keys (sorted):\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &keys[i]);

   printf("Enter frequencies:\n");
   for (int i = 0; i < n; i++)
      scanf("%d", &freq[i]);

   int cost[100][100];

   // Initialize for single keys
```

```
    for (int i = 0; i < n; i++)
        cost[i][i] = freq[i];

    // Build table
    for (int L = 2; L <= n; L++) // L = chain length
    {
        for (int i = 0; i <= n - L; i++)
        {
            int j = i + L - 1;
            cost[i][j] = 99999;

            int fsum = sum(freq, i, j);

            for (int r = i; r <= j; r++)
            {
                int c = ((r > i) ? cost[i][r - 1] : 0) +
                        ((r < j) ? cost[r + 1][j] : 0) +
                        fsum;

                if (c < cost[i][j])
                    cost[i][j] = c;
            }
        }
    }

    printf("Minimum cost of OBST = %d", cost[0][n - 1]);

    return 0;
}
```